



Chat shared

Share domain vocabulary, utilities and common UI primitives.

Problem and boundary

Public contract

Working integration

@epam/ai-dial-chat-shared



When this library is useful

Share domain vocabulary, utilities and common UI primitives.

01

Agree on shapes

Use the same message, stage and attachment types.

02

Reuse common UI

Share Markdown, icons and resource summaries.

03

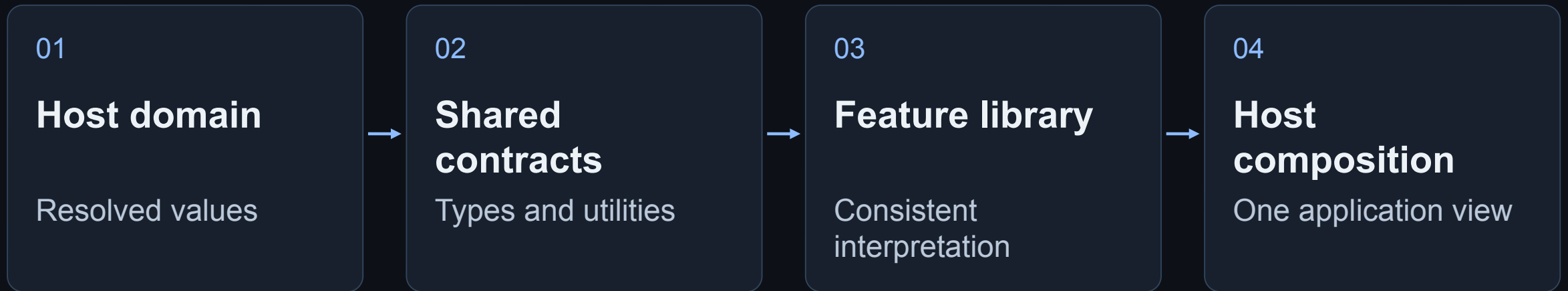
Bridge file managers

Use a typed controller without host persistence knowledge.



Follow the ownership boundary

Inputs and effects stay explicit.



Library: Shared models, pure utilities, UI primitives and narrow file-manager event bindings.



The public contract to start with

`buildCssVars` · selected surface; full API index is in the bundle.

Input or operation	Contract
MessageRole / Stage	Shared enums and domain records
<code>buildCssVars(vars)</code>	CSSProperties, omitting empty values
<code>/markdown</code>	MarkdownRenderer and related exports
<code>/file-manager</code>	Shell, controller types and bindings



A minimal workspace integration

TypeScript / React example · imports and host setup abridged.

```
const style = buildCssVars({
  '--text-primary': '#EDF2F7',
  '--unused': undefined,
});
// Undefined values are omitted.
const role = MessageRole.Assistant;
```

Full example: [examples/chat-shared.tsx](#) · uses [@epam/ai-dial-chat-shared](#)



Wire a realistic host workflow

Keep chat shared focused on its public responsibility.

01

Resolve host values

Choose the text and theme values in the application.

02

Render shared content

Use MarkdownRenderer and buildCssVars in a host view.

03

Keep integration local

Wire file-manager callbacks and persistence in the host.



Reuse the Markdown entry

A common renderer keeps display behavior consistent.

```
import { MarkdownRenderer }  
  from '@epam/ai-dial-chat-shared/markdown';  
  
<MarkdownRenderer  
  content="**Demo:** shared Markdown rendering"  
>
```

Focused source-backed example · see [examples/](#) and [content/api-index.md](#).



Available customization

01

Focused entries

`/markdown` and `/file-manager` expose narrower entry points.

02

CSS variable builder

`buildCssVars` drops undefined and empty values.

03

Canonical grid hook

`useGridEditingScroll` lives here and is re-exported by `chat-hooks`.



Find the integration and its checks

Existing tests are evidence of intended behavior; suite results are not claimed.

Evidence	Where to look
Real consumer	<code>prompt-editor/src/components/PromptEditor/PromptEditor.tsx</code>
Test evidence	disables the Attach button when no paths are selected
Test evidence	calls <code>ensureIndexVisible</code> with the row index when inline rename starts



Constraints that affect integration

01

More than types

The source includes runtime utilities and rendered UI.

02

Entry-specific peers

The file-manager entry needs its optional peer pair installed.

03

Legacy boundary assumptions

The old no-dependencies description is not the current graph.